

Reporte Semanal

Nombre:	Miguel de Jesús Degollado Macías
Matrícula:	A01255388
Programa Académico:	Ingeniería en Tecnologías Computacionales — Tecnológico de Monterrey
Empresa:	CENACE (Centro Nacional de Control de Energía) — Hermosillo, Sonora
Tutor académico:	Mario Duran Vega
Fecha:	28 de septiembre al 2 de octubre de 2026 (Semana 8)

1. Descripción de la fase u objetivos desarrollados en la semana declarados en el cronograma.

El objetivo de esta semana fue concluir la vinculación de las partidas presupuestales con sus viáticos, pendiente de la semana anterior, y fusionar la información de viáticos con el Reporte General, de modo que cada reporte general cuente con su propio reporte de viáticos, importable y reimportable desde el mismo. Adicionalmente, se atendieron ajustes de interfaz solicitados por el Product Owner (sección de gráficas colapsable y filtros reubicados debajo de ella) y se inició la automatización de la validación del código en Azure DevOps, con el flujo Pull Request → Pipeline → Publicación en Dev como meta.

2. Descripción del desarrollo de las actividades semanales (metodología, recursos, procedimientos que se siguieron)

El desarrollo de esta semana se organizó en cinco frentes:

- **Vinculación de partidas con viáticos:** con base en el archivo de ejemplo del Product Owner, que definió como dato en común el Doc. Ref de la factura y el Doc. Comprobación del viático, el detalle de cada partida muestra ahora, junto a cada factura, los datos del viático correspondiente (status, doc. anticipo, fechas de inicio, final y comprobación, transporte, tipo de comisión, anticipo y días de viáticos). Los códigos se tradujeron con los catálogos de SAP y con el catálogo de transportes existente; no se exponen nombre ni RPE del empleado, al no haber sido solicitados, y las facturas sin viático conservan su vista original.
- **Fusión de Viáticos con Reporte General:** cada reporte de viáticos se liga uno a uno con su Reporte General mediante una nueva llave. Dentro del Reporte General se agregó un botón para importar los viáticos que, una vez importados, cambia a reimportar (con opción de reemplazar la información, capacidad que no existía para viáticos) y permite abrir el reporte ligado. El contenedor se crea de forma automática con el nombre del Reporte General, el botón Regresar vuelve a él, se retiró Viáticos del menú principal y se eliminó la creación manual de reportes de viáticos. El listado se conservó como vista de auditoría, con el estado de cada reporte (sin Reporte General, vacío o con su total de viáticos).
- **Ajustes de interfaz del reporte general:** las gráficas y los indicadores clave se agruparon en una sección colapsable, y el buscador, los filtros de austeridad y capacitación y la leyenda se reubicaron debajo de ella, justo antes de la tabla, sin modificar la lógica de las gráficas, que sigue respondiendo a los mismos filtros.
- **Corrección de errores:** se identificó que al entrar a la pantalla de importación se creaba un reporte de viáticos aunque no se importara ningún archivo, lo que dejaba reportes vacíos ligados al Reporte General; la creación se movió al momento de importar un archivo válido, con reversión del contenedor si la inserción falla. Además, el total de registros del listado nunca se calculaba, por lo que se implementó para poder identificar los reportes vacíos.
- **Automatización en Azure DevOps:** se revisaron los pipelines existentes (tres de integración que construyen las imágenes Docker de la Web y de las dos APIs, y uno de liberación) y se confirmó que la publicación al ambiente de pruebas se realiza de forma manual. Se definió un pipeline de validación que compila la solución completa en cada Pull Request hacia dev, junto con su configuración como

política de rama para impedir la fusión si la compilación falla; la publicación automática queda pendiente de documentar el proceso manual actual.

```
$ find . -type f \( -path "**GetByComprobacion*" -o -path "**GetByReporteGeneral*" \) | sort
ReportesViaticos/Queries/GetByReporteGeneral/GetReporteViaticoByReporteGeneralQuery.cs
ReportesViaticos/Queries/GetByReporteGeneral/GetReporteViaticoByReporteGeneralQueryHandler.cs
Viaticos/Queries/GetByComprobacion/GetViaticosByComprobacionQuery.cs
Viaticos/Queries/GetByComprobacion/GetViaticosByComprobacionQueryHandler.cs

$ find . \( -name "ViaticoFacturaVM.cs" -o -name "ViaticoCatalogos.cs" -o -name "ImportViaticos.cshtml" \) | sort
SGP.Models/ViewModels/ViaticoFacturaVM.cs
SGP.Web/Areas/Presupuesto/Views/ReporteGeneral/ImportViaticos.cshtml
SGP.Web/Utils/ViaticoCatalogos.cs
```

Archivos nuevos de las historias entregadas esta semana (vinculación de viáticos con facturas y fusión con Reporte General), verificados en el proyecto

3. Listado de las actividades siguientes cercanas o pendientes no resueltas en la semana anterior, así como la justificación de su incumplimiento.

La vinculación de partidas con viáticos, pendiente de la semana anterior, se concluyó una vez que el Product Owner definió el dato en común entre ambos reportes (Doc. Comprobación del viático y Doc. Ref de la factura), información que no existía en el modelo y cuya ausencia justificó no iniciar el desarrollo. Quedan pendientes: (1) la publicación automática al ambiente de pruebas, que requiere conocer los pasos exactos del despliegue manual actual (servidor, método de conexión y comandos) y que se consultará con el asesor de practicas y con TI; y (2) la depuración de los reportes de viáticos vacíos creados antes de la corrección, identificables en la pantalla de auditoría.

4. Autorreflexión personal, mencionando problemas enfrentados, aprendizajes, temas personales o profesionales relacionados con su desempeño.

Esta semana aprendí que la relación entre dos fuentes de datos debe confirmarse con quien las conoce antes de programarla: al contar con el dato en común validado por el Product Owner (Doc. Comprobación del viático contra Doc. Ref de la factura) pude comprobar el cruce con el archivo real, en lugar de basarme en suposiciones. También aprendí que una consulta de lectura nunca debe crear información: al crear el reporte de viáticos solo por visitar la pantalla de importación dejé reportes vacíos que aparentaban estar importados; la corrección fue crear los datos únicamente cuando la operación se completa y revertirlos si falla. Comprendí además que agregar un campo a una entidad existente deja los registros anteriores sin ese dato, por lo que previene enlaces rotos con un respaldo y una pantalla de auditoría para identificarlos. Finalmente, al revisar los pipelines de Azure DevOps aprendí a verificar lo que realmente hace un proceso antes de automatizarlo: los nombres sugerían un ambiente (un repositorio de imágenes de QA y una etapa llamada Producción) y solo al revisar el contenido confirmé que el despliegue a pruebas es manual y que primero debo documentarlo.

Evidencias: Pipelines de integración existentes del proyecto en Azure DevOps, revisados para definir la automatización del flujo Pull Request → Pipeline → Publicación en Dev

Pipelines	
Recent All Runs	
Recently run pipelines	
Pipeline	Last run
 SGP-Docker container-CI-Web	#20240917.4 • Fix en URL de descarga de factura Manually run by  maín
 SGP-Docker container-CI-API-Pagos	#20240611.1 • Captura de NC Manually run by  maín
 SGP-Docker container-CI-API-Presupue...	#20240416.2 • Fix temporal para el borrado de periodos (deshabilitar paginación) Manually run by  maín